ESEN · INGENIERÍA DE SOFTWARE Y NEGOCIOS DIGITALES

Metodologías Ágiles de Desarrollo · PN0913 · Sesión 23

Taller: la deuda técnica de su propio proyecto

Jueves 22 de octubre de 2026 · trabajo en equipo · 100 minutos

Equipo:	Integrantes presentes:
AVERGUERS	Iván Alessandro Vásquez Martínez Kevin Giovanni Landos Castaneda José Mario Sandoval Rodríguez Mariela Nicole Flores Arévalo Víctor Sebastián Quijada

Hoy trabajan sin clase expositiva

Todo lo que necesitan saber está en la Parte 1. Léanla en equipo, en voz alta si ayuda, y después trabajen sobre su propio repositorio. No es un ejercicio teórico: lo que decidan hoy entra al Sprint 4.

- Necesitan: el repositorio abierto, el tablero del equipo y los ADRs que escribieron el martes.
- Respeten los tiempos de cada parte. Si se atrasan en una, recórtenla: la Parte 4 es la que no se puede saltar.
- Al final entregan esta guía completa más lo que hayan agregado al backlog.

PARTE 1 · 15 minutos

Lean esto antes de empezar

Qué es la deuda técnica

Ward Cunningham, uno de los firmantes del Manifiesto Ágil, propuso la metáfora en 1992: entregar código que no está del todo bien es como pedir un préstamo. Se puede hacer, y a veces conviene, porque permite entregar antes. Pero se pagan intereses: cada cambio futuro en esa zona cuesta un poco más de lo que habría costado.

La deuda no es mala en sí misma. Casi todo equipo que entrega a tiempo tiene deuda. Lo que sí es grave es tener deuda sin saberlo, porque entonces nadie puede decidir si conviene pagarla.

Deuda no es lo mismo que desorden

La diferencia práctica

- Deuda: una decisión consciente que compró algo. «Dejamos la validación incompleta para llegar a la Review del viernes.» Se puede explicar qué se ganó.

• Desorden: simplemente no se hizo bien. «Nadie sabe por qué ese archivo tiene 600 líneas.» No compró nada, no tiene justificación.

La prueba: si pueden decir qué ganaron a cambio, es deuda. Si no, es desorden. Y el desorden no se negocia: se limpia.

Los cuatro tipos

Martin Fowler propuso clasificar la deuda según dos ejes: si fue deliberada o accidental, y si fue prudente o imprudente. Van a usar esta clasificación en la Parte 2.

	Prudente	Imprudente
Deliberada	«Sabemos que no es la mejor forma, lo hacemos para llegar a la fecha, y sabemos qué nos va a costar después.»	«No hay tiempo para pensar el diseño.» Se decidió, pero sin medir la consecuencia.
Accidental	«Ahora que terminamos, vemos cómo deberíamos haberlo hecho.» Es la deuda que produce aprender.	«¿Qué es separar la lógica de la interfaz?» No se sabía que había una decisión que tomar.

Cómo se pagan los intereses

El interés no se paga en cuotas fijas: se paga cada vez que alguien toca esa zona del código. Una parte fea que nadie va a volver a tocar tiene interés cero, y pagarla sería desperdicio. Una parte fea que se toca cada sprint cobra interés constantemente.

De ahí la regla que van a aplicar hoy: no se paga toda la deuda, se paga la que cobra intereses.

Refactorizar

Refactorizar es cambiar la estructura interna del código sin cambiar lo que hace. Si cambia el comportamiento, no es refactorización: es otra cosa.

Requiere pruebas. Sin pruebas que confirmen que el comportamiento no cambió, no están refactorizando: están reescribiendo y esperando que salga bien.

Se hace en pequeño y de forma continua, dentro del trabajo normal. Un «sprint de refactorización» suele ser la señal de que se dejó crecer demasiado.

La regla del campamento: dejen el código un poco mejor de como lo encontraron. No perfecto, solo un poco mejor.

PARTE 2 · 30 minutos

Inventario: busquen su deuda

Abran su repositorio y recorran el código que escribieron en los sprints 1 a 3. Busquen señales de la lista y anoten entre cinco y ocho hallazgos reales. No inventen: todos tienen que poder señalarse con un archivo.

Qué buscar · catálogo de señales

- El mismo código, o casi el mismo, aparece en dos o tres lugares.
- Un archivo o una función demasiado larga para leerla de una sentada.
- Nombres que no dicen qué hace la cosa: datos, manejar, procesar, temp.
- La lógica del negocio está mezclada con la interfaz o con el acceso a datos.
- Para hacer un cambio pequeño hay que tocar tres lugares a la vez.
- Código que ya no se usa y nadie se atrevió a borrar.
- Valores repetidos a mano: URLs, claves, números que significan algo.
- Configuración escrita dentro del código en lugar de estar aparte.
- Zonas críticas sin ninguna prueba que las cubra.
- El archivo que nadie del equipo quiere abrir. Ese siempre cuenta.

#	Qué encontraron y dónde (archivo o zona)	¿Deuda o desorden?	Si es deuda: ¿qué tipo?
1	Backend con demasiadas responsabilidades. En backend/app.mjs, de aproximadamente 550 líneas, están las rutas HTTP, validaciones, reglas de contratos y pagos, consultas SQL y entrega de archivos. Esto dificulta modificar una funcionalidad sin revisar código de otras. Ejemplos: validateContract, saveContract, savePayment y createServer.	Desorden. No encontramos una justificación documentada para concentrar todas esas responsabilidades.	No aplica.
2	Lógica de cálculo mezclada con la interfaz. En frontend/app.js, de aproximadamente 609 líneas, se manejan el catálogo, la selección, el formulario y las solicitudes al servidor. En updateEstimate, por ejemplo, se calculan tarifas y totales mientras se modifican elementos de la página. Esto dificulta probar los cálculos por separado.	Desorden. La mezcla de responsabilidades aumenta la dificultad de mantenimiento.	No aplica.

#	Qué encontraron y dónde (archivo o zona)	¿Deuda o desorden?	Si es deuda: ¿qué tipo?
3	Opciones del contrato repetidas manualmente. Los códigos de seguros y formas de pago aparecen en backend/contrato-legal.mjs y en los selectores de frontend/index.html. Agregar una opción exige actualizar ambos lugares; si se olvida uno, la interfaz y el servidor quedan descoordinados	Desorden. No hay una única fuente para definir las opciones disponibles.	No aplica.
4	Duración de la sesión definida dos veces. En backend/auth.mjs, la cookie utiliza SESSION_SECONDS = 28_800, pero el vencimiento en la base se escribe aparte como INTERVAL '8 hours'. Actualmente coinciden, pero cambiar solo uno produciría duraciones diferentes.	Desorden. El mismo valor de configuración se mantiene en dos representaciones independientes.	No aplica.
5	Pruebas de reservas y pagos sin comprobar bloqueos entre conexiones reales de PostgreSQL. backend/app.test.mjs y backend/cobros.test.mjs utilizan PGlite. Existen pruebas concurrentes, pero el ADR-002 reconoce que no verifican los bloqueos entre conexiones reales.	Deuda. Se conserva un entorno de pruebas local sencillo, aceptando una diferencia respecto a producción.	Deliberada prudente. La limitación se reconoce y documenta en el ADR-002.
6	Preparación de la base dependiente de un paso manual antes de publicar. En backend/server.mjs, PostgreSQL se verifica al arrancar, pero sus cambios de esquema deben aplicarse previamente mediante scripts. El ADR-002 reconoce que olvidar este paso puede impedir el arranque.	Deuda. Se mantiene el control explícito sobre los cambios en la base, a cambio de depender de una coordinación manual con el despliegue.	Deliberada prudente. Es una limitación conocida y documentada.

PARTE 3 · 25 minutos

¿Cuánto cuesta cargarla?

Para cada hallazgo de la Parte 2 decidan dos cosas: con qué frecuencia les cobra intereses, y cuánto costaría pagarla. De ahí sale la decisión.

Cómo decidir

- Interés alto + pago barato → se paga ya. Entra al Sprint 4.
- Interés alto + pago caro → se paga por partes, o se aísla para que deje de cobrar. Discutan cuál de las dos.
- Interés bajo + pago barato → si sobra tiempo. No antes de lo anterior.
- Interés bajo + pago caro → se acepta y se documenta. No se toca.

Interés alto significa: lo tocamos casi cada sprint, o nos ha roto algo, o nos frena para terminar el proyecto.

Las frecuencias y las horas son estimaciones de planificación según las zonas afectadas; no son tiempos medidos. Los pagos parciales de los hallazgos 1, 2 y 5 reducen el problema, pero no lo cierran.

#	¿Cada cuánto nos cobra? (alto / medio / bajo)	¿Qué costaría pagarla? (horas aproximadas)	Decisión (pagar ya / por partes / aceptar)
1	Alto. Al modificar contratos, reservas o pagos se debe revisar un archivo que reúne reglas, SQL y rutas.	12 h en total. Primera extracción acotada de validaciones de contratos: 3 h, incluidas regresiones.	Por partes. Separar primero validateContract, conservando la API y sus exportaciones. Posponer la reorganización completa.
2	Medio. Cada ajuste de fechas, selección de máquinas o totales obliga a trabajar con cálculos y DOM juntos.	8 h en total. Extraer el cálculo del estimado y probarlo: 2 h.	Por partes. Aislar el cálculo de días, tarifa conjunta y total; mantener la presentación en updateEstimate.
3	Medio. Cada alta o cambio de seguro o forma de pago exige coordinar el HTML con el servidor.	3 h para una definición compartida, su integración y pruebas de coherencia.	Pagar ya. Centralizar los códigos y conservar textos propios para la interfaz y el contrato legal.
4	Bajo. Cobra al cambiar la duración de la sesión; hoy ambas definiciones equivalen a ocho horas.	1 h para reutilizar SESSION_SECONDS y comprobar cookie y vencimiento.	Pagar ya, en última prioridad. El cambio es pequeño y evita que la sesión del servidor y la cookie se descoordinen.
5	Alto. Cada cambio en reservas o pagos deja sin comprobar los bloqueos entre conexiones reales; afecta disponibilidad y saldos.	10 h para ampliar la cobertura e integrarla a CI. Primer tramo con los dos casos críticos: 5 h.	Por partes. Probar reservas solapadas y pagos simultáneos con dos conexiones PostgreSQL reales. Mantener PGlite.

#	¿Cada cuánto nos cobra? (alto / medio / bajo)	¿Qué costaría pagarla? (horas aproximadas)	Decisión (pagar ya / por partes / aceptar)
6	Bajo mientras no cambie el esquema. Cobra en los despliegues que requieren tablas o restricciones nuevas.	8 h para automatizar migraciones con control de versiones y validar fallos y compatibilidad.	Aceptar temporalmente. Mantener la preparación explícita y verificar la base antes de publicar. Revisar si aumentan los cambios de esquema.

Las horas son estimaciones iniciales de esfuerzo según el alcance técnico, debemos validarlas con quienes harán las tareas del sprint 4 y ajustarlas según nuestra experiencia y conocimiento de desarrollo ·

El plan: qué entra al Sprint 4

Esta es la parte que no se puede saltar. Lo que decidan aquí es trabajo real del equipo, no un ejercicio.

A · Lo que se paga

Conviertan cada ítem de «pagar ya» o «por partes» en una tarjeta del Sprint 4, en su tablero. Escriban aquí cuáles crearon.

Las cuatro tarjetas quedan redactadas para incorporarlas al tablero del Sprint 4. Los códigos DT-S4 son identificadores de este plan.

Tarjeta propuesta para el tablero	¿Qué mejora concretamente?	Horas estimadas
DT-S4-01 Separar validaciones del contrato y cálculo del estimado Hallazgos 1 y 2	Extraer validateContract a un módulo y los cálculos de frontend/app.js a funciones sin DOM. Mantener rutas, errores y exportaciones existentes. Cierre: pasan las pruebas de contratos y se comprueban un día, varias máquinas y fechas inválidas; la selección y el total visible conservan su comportamiento.	5 h 3 backend 2 frontend
DT-S4-02 Centralizar seguros y formas de pago Hallazgo 3	Usar una definición compartida para los códigos válidos del backend y los selectores. Conservar las etiquetas breves de pantalla y el texto legal. Cierre: cada opción mostrada es aceptada por el servidor; los códigos desconocidos se rechazan y el PDF mantiene sus cláusulas.	3 h

Tarjeta propuesta para el tablero	¿Qué mejora concretamente?	Horas estimadas
DT-S4-03 Unificar la duración de la sesión Hallazgo 4	Calcular Max-Age y expira_en desde SESSION_SECONDS en backend/auth.mjs mediante SQL parametrizado. Conservar 28 800 segundos. Cierre: la prueba del login compara la duración de la cookie con el vencimiento guardado y las pruebas de autenticación siguen pasando.	1 h
DT-S4-04 Comprobar concurrencia real en reservas y pagos Hallazgo 5	Preparar PostgreSQL de pruebas aislado y usar dos conexiones independientes. Coordinar transacciones solapadas para ejercer el bloqueo. Cierre: solo se confirma una reserva del mismo equipo y período; dos pagos que juntos exceden el saldo no causan sobrepago. Registrar resultados y comando reproducible. La integración a CI queda para el siguiente tramo.	5 h

Esfuerzo previsto: 14 horas-persona, incluidas pruebas y revisión. Prioridad: DT-S4-04, DT-S4-01, DT-S4-02 y DT-S4-03. El equipo debe contrastar estas horas con su capacidad y las historias pendientes; no se presupone un sprint completo de limpieza.

B · Lo que se acepta

El ADR se entrega por separado en ADR-005.md. Contiene el contexto, la decisión y las consecuencias de aceptar temporalmente la preparación manual de PostgreSQL. Agregado en el archivo de los ADR en el repositorio.

PARTE 5 · 10 minutos

Intercambio con otro equipo

Júntense con otro equipo y muéstrense los inventarios. Cada equipo responde dos preguntas sobre el del otro:

¿Hay algún hallazgo que clasificaron como deuda y que en realidad es desorden? Es decir: ¿pueden explicar qué ganaron a cambio?

¿Hay algo que decidieron aceptar y que al otro equipo le parece que les va a cobrar intereses antes de la Feria?

Lo que nos señaló el otro equipo	¿Cambiamos algo por eso? ¿Qué?